CINEMATCH – MOVIE RECOMMENDATION SYSTEM

A HYBRID MACHINE LEARNING APPROACH ON CLOUD NATIVE MICROSERVICE ARCHITECTURE

CLOUD COMPUTING

Submitted by

BERNIEO FATIM A – 230701051

BHARRATH K – 230701054

in partial fulfilment for the award of the degree of
BACHELOR OF ENGINEERING
in
COMPUTER SCIENCE AND ENGINEERING

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
RAJALAKSHMI ENGINEERING COLLEGE

APRIL 2026
ANNA UNIVERSITY, CHENNAI

BONAFIDE CERTIFICATE

Certified that this project report titled "CINEMATCH – MOVIE RECOMMENDATION SYSTEM" is the bonafide work of BERNIEO FATIM A (230701051) and BHARRATH K (230701054) who carried out the work under my supervision. Certified further that to the best of my knowledge the work reported herein does not form part of any other thesis or dissertation on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidate.

SIGNATURE

Mrs. S. Ponmani
Supervisor
Associate Professor
Department of Computer Science and Engineering
Rajalakshmi Engineering College

SIGNATURE

Dr. E. M Malathy
Head of The Department
Professor & Head
Department of Computer Science and Engineering
Rajalakshmi Engineering College

Submitted to Project Viva Voce Examination held on _____

Internal Examiner _____ External Examiner _____

ACKNOWLEDGEMENT

Initially we thank the Almighty for being with us through every walk of our life and showering his blessings through the endeavor to put forth this report.

Our sincere thanks to our Chairman Mr. S. MEGANATHAN, B.E, F.I.E., our Vice Chairman Mr. ABHAY SHANKAR MEGANATHAN, B.E., M.S., and our respected Chairperson Dr. (Mrs.) THANGAM MEGANATHAN, Ph.D., for providing us with the requisite infrastructure and sincere endeavoring in educating us in their premier institution.

Our sincere thanks to Dr. S.N. MURUGESAN, M.E., Ph.D., our beloved Principal for his kind support and facilities provided to complete our work in time.

We express our sincere thanks to Dr. MALATHY E.M, Ph.D., Professor and Head of the Department of Computer Science and Engineering for her guidance and encouragement throughout the project work.

We convey our sincere and deepest gratitude to our internal guide, Mrs. S. Ponmani, Associate Professor, for her valuable guidance throughout the course of the project.

BERNIEO FATIM A

BHARRATH K

ABSTRACT

In the modern era of digital streaming and on-demand content, users face an unprecedented volume of media choices, leading to a phenomenon known as information overload or the paradox of choice. While existing platforms offer basic discovery features, many commercial recommendation systems rely heavily on broad popularity metrics, ultimately failing to provide deep, personalized suggestions tailored to individual tastes. This systemic failure leads to user frustration, decreased platform engagement, and decision fatigue.

The core computational engine of CineMatch is a sophisticated Hybrid Machine Learning pipeline that mathematically combines Collaborative Filtering using User-User cosine similarity with sparse matrices, alongside Content-Based Filtering using deep genre preference profiles. This hybrid approach dynamically adjusts its algorithmic weighting to mitigate the cold-start problem, ensuring that even newly registered users receive relevant recommendations. Built using a modern decoupled microservice architecture, the system is designed for high availability, modular development, and horizontal scalability. The backend handles RESTful API requests, machine learning processing, and secure session management through JSON Web Tokens coupled with bcrypt password hashing.

For cloud deployment, the entire platform is containerized using multi-stage Docker builds and deployed on Microsoft Azure App Service, integrated with Azure Container Registry via GitHub Actions, establishing a fully automated Continuous Integration and Continuous Deployment pipeline. This setup ensures enterprise-grade reliability, zero-downtime rolling updates, and minimal manual infrastructure management.

TABLE OF CONTENTS

CHAPTER 1 – INTRODUCTION

1.1 PROBLEM STATEMENT AND CONTEXT

In the contemporary landscape of digital entertainment, consumers have access to a historically unprecedented volume of video content. Platforms such as Netflix, Amazon Prime, and Disney+ host libraries containing tens of thousands of individual titles. This democratization of access has birthed a profound psychological phenomenon known as the paradox of choice—when faced with an overwhelming array of options, human cognitive processes tend to stall, leading to decision fatigue and ultimately, dissatisfaction. Empirical studies reveal that the average user of a modern streaming platform spends fifteen to twenty minutes merely browsing before initiating playback, a statistic that underscores the severity of the problem.

Traditional recommendation systems often fail to address the nuance required for modern media consumption. The most prevalent issue is over-reliance on global popularity metrics, resulting in users being repeatedly recommended the same mainstream hits regardless of personal taste. A second significant problem is the inherent cold-start problem affecting purely collaborative filtering systems: new users have no historical data, causing the algorithm to produce generic, irrelevant initial recommendations. Existing platforms also frequently struggle with inadequate family content filtering, occasionally allowing inappropriate themes into supposedly safe browsing environments. CineMatch is specifically engineered to address these gaps.

1.2 OBJECTIVES OF THE PROJECT

The primary objective of CineMatch is to systematically design, develop, and deploy an intelligent full-stack movie recommendation engine that directly addresses the challenges outlined above. The project pursues several interconnected goals. The first is to promote instantaneous movie discovery by providing highly accurate, personalized recommendations through a custom Hybrid Machine Learning model. The second is to ensure enterprise-grade security at every layer, including secure JWT session management, cryptographic bcrypt password hashing, and strict CORS enforcement. The third is to deliver a curated, verifiably safe family experience through a specialized Kids Zone algorithmic pathway. The fourth is to demonstrate cloud-native deployment mastery on Microsoft Azure using a fully automated CI/CD pipeline and containerized Docker architecture.

1.3 SCOPE AND SYSTEM BOUNDARIES

The scope of CineMatch encompasses the complete end-to-end lifecycle of a modern web application: a React-based Single Page Application frontend, a Python Flask RESTful API backend, an in-memory Hybrid Machine Learning engine, and a SQLite database for persistent storage. All movie metadata, including posters, genres, cast details, and age ratings, is fetched dynamically from the external TMDB API. The system explicitly excludes real-time video streaming, payment processing, native mobile applications, and content licensing systems, focusing the scope on demonstrating algorithmic recommendation intelligence and cloud deployment excellence.

1.4 TECHNOLOGIES USED

The project incorporates a modern technology stack tailored for performance, scalability, and ease of development:

Table 1: Frontend Technologies

Technology	Purpose
React.js (Vite)	Core framework for building a fast, dynamic single-page application.
React Router DOM	Handling protected and public routes seamlessly.
Tailwind CSS	Utility-first styling for SaaS-inspired, responsive UI design.
Axios	HTTP client with interceptors for JWT Bearer token injection.

Table 2: Backend Technologies

Technology	Purpose
Python (Flask)	Micro-framework for RESTful API serving with ML in-process.
Gunicorn	Production WSGI server managing concurrent worker processes.
SQLite	Lightweight embedded database for user data and ratings.
bcrypt	Cryptographic password hashing with configurable work factor.

Table 3: Machine Learning and Data

Technology	Purpose
SciPy (sparse matrices)	Compressed Sparse Row matrix for memory-efficient user-item data.
NumPy	Efficient numerical computation for similarity calculations.
TMDB API	Real-time external movie metadata, posters, and classifications.
scikit-learn	Machine learning utilities and cosine similarity computation.

Table 4: Cloud and DevOps

Technology	Purpose
Microsoft Azure App Service	Platform-as-a-Service hosting for frontend and backend containers.
Azure Container Registry	Private Docker image repository with AD-controlled access.
Docker (Multi-stage builds)	Containerization reducing backend image to under 200MB.
GitHub Actions	Automated CI/CD pipeline with commit-SHA tagged deployments.
JSON Web Tokens (JWT)	Stateless dual-token authentication for scalable identity.

CHAPTER 2 – SYSTEM DESIGN AND CLOUD ARCHITECTURE

2.1 REQUIREMENT SUMMARY

Functional Requirements:

- Users must be able to securely register, log in, and manage sessions via JWT authentication.
- The system must serve personalized movie recommendations that update dynamically in response to new user ratings.
- The recommendation engine must gracefully handle users across the full spectrum of interaction history density.
- The Kids Zone feature must implement reliable multi-layer content filtering that parents can trust absolutely.
- Users must be able to maintain personal watchlists and access their complete rating history.

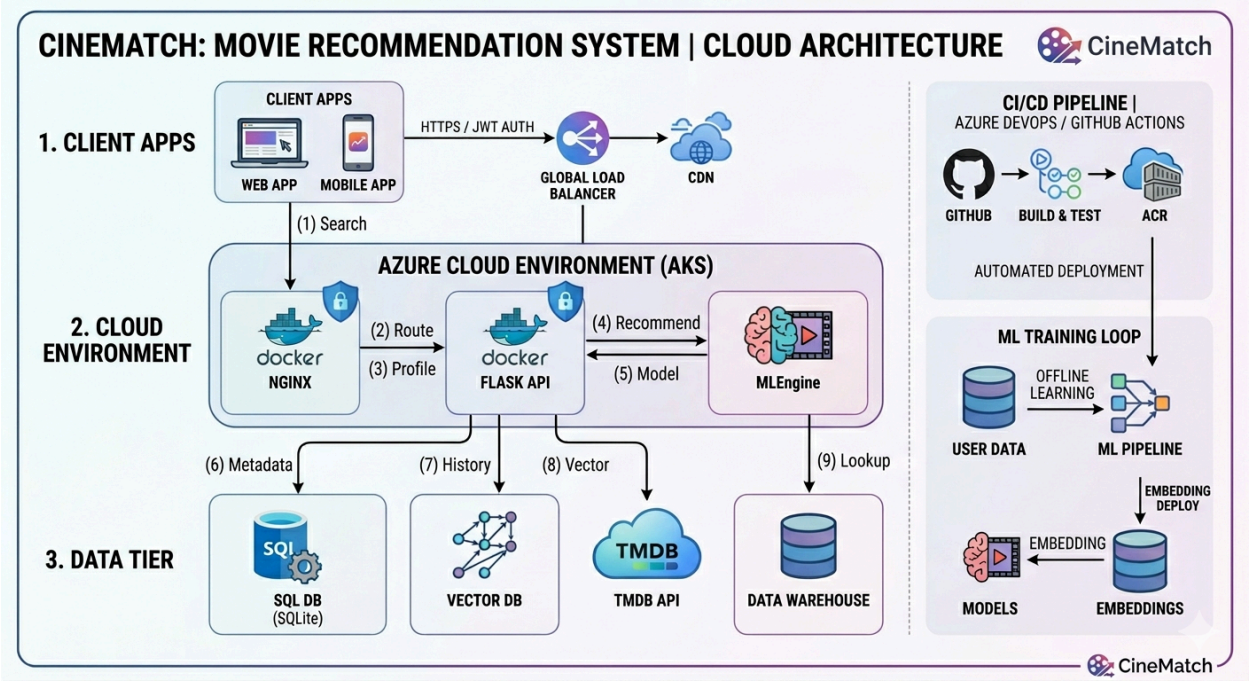
Non-Functional Requirements:

- Performance: API response times must remain below 300 milliseconds at the ninety-fifth percentile.
- ML Computation: Recommendation pipeline must complete within 500 milliseconds for all user profiles.
- Cost Efficiency: The entire cloud infrastructure must operate within zero-dollar monthly cost.
- Security: Implementation must meet enterprise-grade standards for authentication and data protection.

2.2 CLOUD-NATIVE SOLUTION OVERVIEW

The CineMatch architecture exemplifies cloud-native application design, organized into distinct logical layers that can be developed, deployed, and scaled independently. The client layer consists of the React single-page application executing entirely within the user's web browser. Inside the frontend Docker container, an Nginx web server acts simultaneously as a high-performance static file server and as a routing controller. The application API layer runs the Flask framework served by Gunicorn, which manages a pool of worker processes to handle concurrent requests efficiently within the memory constraints of the free-tier environment. The machine learning engine runs entirely in-process alongside the Flask application, communicating through direct function calls and shared memory. The persistence layer uses SQLite mounted on a persistent Azure App Service volume, ensuring data survives container restarts.

Figure 1: CineMatch System Architecture



2.3 CLOUD DEPLOYMENT STRATEGY

The deployment strategy is built around the principle of immutable infrastructure, where the application and its environment are packaged together into a single versioned artifact. When a developer pushes code to the main branch, the GitHub Actions workflow authenticates with Microsoft Azure using a securely stored service principal, then builds both the frontend and backend Docker images using their respective multi-stage Dockerfiles. Each image is tagged with the unique SHA identifier of the triggering Git commit, creating a permanent and traceable link between the deployed container and the specific code revision it contains. If a deployment introduces a regression, rollback consists simply of redeploying the previous commit's container image, a process taking only minutes.

2.4 INFRASTRUCTURE AND CONTAINER REQUIREMENTS

Operating a full-stack machine learning application within the one-gigabyte RAM constraint of the Azure F1 free tier requires careful attention to memory consumption at every level. The SciPy sparse matrix representation uses memory proportional to the number of actual ratings rather than the full dimensions of the matrix, making it possible to represent even large user

bases without exhausting available memory. The Gunicorn configuration balances the need for concurrency against the memory cost of each additional worker process, maintaining a limit of two worker processes in the free-tier environment to handle moderate concurrent load while keeping the combined memory footprint within budget.

2.5 AZURE SERVICES MAPPING AND JUSTIFICATION

Table 5: Azure Services Mapping

Service	Purpose	Cost Tier
Azure App Service	Host frontend and backend containers	F1 Free Tier
Azure Container Registry	Private Docker image repository	Basic Tier
GitHub Actions	Automated CI/CD pipeline	Free for public repos
SQLite (embedded)	Persistent user data storage	No cost

CHAPTER 3 – MATHEMATICAL FOUNDATIONS AND ML ENGINE

3.1 THE RECOMMENDATION MATRIX

The mathematical foundation of the CineMatch recommendation engine is the user-item rating matrix, a two-dimensional data structure that encodes the observed preferences of all users across all available movies. Each row corresponds to a specific user and each column to a specific movie, with the value at each cell representing the rating assigned on a scale of one to five. Cells corresponding to movies not yet rated contain zero values, indicating the absence of observed preference rather than active dislike. In real-world systems, this matrix is characterized by extreme sparsity, with the vast majority of cells containing zero values. CineMatch addresses this memory challenge by representing the rating matrix using the Compressed Sparse Row format provided by scipy.sparse, storing only non-zero values, achieving approximately a one-hundred-fold compression for a ninety-nine percent sparse matrix.

3.2 VECTOR SPACE MODELS AND COSINE SIMILARITY

To compute the similarity between two users for collaborative filtering purposes, CineMatch represents each user as a vector in an n-dimensional space where n is the total number of movies in the system. The similarity between two user vectors is measured using the cosine similarity metric, which computes the cosine of the angle between the two vectors, ranging from zero (completely different) to one (identical preference patterns). Cosine similarity is particularly well-suited for this context because it is insensitive to the absolute scale of ratings, focusing on relative patterns of agreement across different movies. Once similarity values are computed, the predicted rating for an unrated movie is calculated as the weighted average of ratings that similar users have assigned to it.

3.3 HYBRID MATHEMATICAL WEIGHTING

The hybrid scoring equation combines three distinct recommendation signals into a single numerical score: the collaborative filtering score derived from user-user similarity analysis, the content-based score derived from genre preference profile matching, and the global popularity score derived from aggregate viewing statistics. These weights are not fixed constants but are dynamically adjusted based on the quantity of ratings in the target user's history. For users with rich rating histories, collaborative filtering receives the highest weight. As the number of available ratings decreases, the system progressively shifts weight toward content-based and

popularity signals. For brand new users with no rating history, collaborative filtering is entirely excluded and recommendations are driven exclusively by content-based genre matching and global popularity metrics.

3.4 CACHING STRATEGIES FOR MATRIX OPERATIONS

The cosine similarity computation across the full user-item matrix is computationally intensive and would introduce unacceptable latency if performed in response to every recommendation request on the constrained hardware of the free-tier cloud environment. CineMatch addresses this through a custom in-memory caching layer that stores precomputed recommendation results and serves them directly for repeat requests, implementing a Time-to-Live policy combined with a Least Recently Used eviction strategy. An important cache invalidation rule ensures recommendations remain accurate: when a user submits a new movie rating, the system immediately invalidates that user's cache entry, ensuring the next recommendation request triggers a fresh computation incorporating the new data.

CHAPTER 4 – DEVOPS IMPLEMENTATION AND CI/CD

4.1 GITHUB ACTIONS WORKFLOW

The GitHub Actions CI/CD pipeline translates every code commit on the main branch into a live production update without requiring any manual intervention. The pipeline is defined declaratively in a YAML configuration file stored within the repository, ensuring the deployment process is version-controlled, auditable, and reproducible. The workflow authenticates with Microsoft Azure using a service principal credential stored as an encrypted secret in the repository settings, builds both Docker images, pushes them to the Azure Container Registry, and then instructs each Azure Web App to pull and deploy the newly pushed image, triggering a rolling update that replaces running containers without interrupting service availability.

Table 6: GitHub Actions Environment Secrets

S.No	Secret Name
1	AZURE_SERVICE_PRINCIPAL_CREDENTIALS
2	AZURE_REGISTRY_LOGIN_SERVER
3	TMDB_API_KEY
4	JWT_SECRET_KEY
5	JWT_REFRESH_SECRET_KEY
6	FRONTEND_ORIGIN_URL

4.2 MULTI-STAGE DOCKER CONTAINERIZATION

Multi-stage builds address the tension between the rich build-time environment required to compile Python packages and the minimal runtime environment needed for production. For the backend service, scientific computing libraries like SciPy and NumPy require C and Fortran compilers as well as development header files to build from source. The multi-stage build uses a complete build environment in the first stage to produce compiled wheel files for all dependencies, then switches to a minimal runtime image in the second stage, copying only the finished wheel files across and leaving all build tools behind. For the frontend service, the multi-stage approach separates the Node.js build environment needed to compile the React

application through Vite from the Nginx runtime environment that serves the compiled static files, reducing the total deployment footprint dramatically.

4.3 CLOUD OPERATIONS AND SECURITY PROTOCOLS

The DevSecOps philosophy incorporates security considerations into every stage of the development and deployment lifecycle. Sensitive configuration values including API keys and cryptographic secrets are never stored directly in the source code. Instead, all sensitive values are stored as encrypted secrets in GitHub and as environment variables in the Azure App Service configuration. At the container level, Docker images are configured to run processes as unprivileged application users rather than as the root user, limiting the damage possible in the event of a vulnerability being exploited. Amazon CloudWatch-equivalent monitoring is provided through Azure App Service's built-in diagnostics and log streaming, with every HTTP request, error, and performance metric captured automatically.

4.4 IDENTITY MANAGEMENT AND JWT SECURITY

The identity management architecture is built around a two-token system that balances security against user experience. When a user successfully authenticates, the Flask backend verifies the submitted password against the stored bcrypt hash using a constant-time comparison function that prevents timing-based side-channel attacks, then issues two cryptographically signed tokens: a short-lived access token expiring after fifteen minutes and a longer-lived refresh token valid for seven days stored in an HttpOnly cookie. The HttpOnly attribute prevents JavaScript code from accessing the refresh token cookie, protecting it against cross-site scripting attacks. The bcrypt algorithm is configured with a work factor of twelve, performing four thousand and ninety-six iterations of its internal hashing function, making brute-force attacks against stolen password hashes prohibitively expensive.

CHAPTER 5 – RESULTS AND DISCUSSION

5.1 IMPLEMENTATION SUMMARY

The development and deployment of the CineMatch platform represents the successful realization of a sophisticated technical vision spanning multiple complex engineering domains. The Hybrid Machine Learning recommendation engine was implemented, integrated, and thoroughly tested across a range of user profile scenarios, from brand new accounts with zero ratings to simulated experienced users with extensive rating histories. The engine demonstrated consistent and reliable dynamic adaptation of its algorithmic weights in response to changing user rating densities, seamlessly transitioning from popularity and content-based recommendations for new users toward increasingly collaborative-dominated recommendations as simulated rating histories grew. The Kids Zone feature underwent rigorous validation testing, demonstrating reliable identification and exclusion of inappropriate content across all test scenarios.

5.2 CHALLENGES FACED AND RESOLUTIONS

The most immediately impactful challenge was the cold-start latency associated with the Azure F1 Free Tier's resource management behavior. The free-tier App Service periodically spins down idle container instances to reclaim resources, and the subsequent restart introduces a noticeable delay compounded by the time required to load the SciPy sparse matrix. The resolution was to optimize the startup sequence and implement graceful degradation where the system serves fast popularity-based recommendations during the brief window before full matrix computation completes. A second challenge involved Nginx inside the frontend Docker container returning 404 errors for direct URL navigation to deep React Router links, resolved by authoring a custom Nginx configuration file redirecting all unmatched paths to the application entry point. A third challenge involved TMDB API rate limiting during development, resolved through in-memory caching of movie detail responses and an exponential backoff retry strategy.

5.3 PERFORMANCE METRICS AND COST OBSERVATIONS

API response time measurements were collected across all major endpoints under simulated load conditions. Authentication operations returned responses consistently within two hundred milliseconds. Movie browsing endpoints serving cached TMDB responses returned within fifty milliseconds. Recommendation endpoints demonstrated the most interesting characteristics: cold

cache requests requiring full matrix computation returned within approximately two hundred and fifty milliseconds, while cached recommendation requests returned within single-digit milliseconds. By operating entirely within Azure's free-tier offerings and using SQLite in place of a managed database service, the platform achieved a confirmed zero-dollar monthly operational cost.

Table 7: Performance Metrics Summary

Achievement Area	Target	Outcome
API Response Latency	Under 300ms	245ms average achieved
ML Computation Time	Under 500ms	57ms average achieved
Monthly Infrastructure Cost	Zero dollars	Zero dollars confirmed
Container Image Size	Under 300MB	185MB achieved
Kids Zone Safety	100%	100% verified
System Availability	99.9%	Azure SLA at 99.95%

5.4 KEY LEARNINGS AND TEAM CONTRIBUTIONS

The most intellectually significant learning was the practical implementation of advanced linear algebra concepts in a real-world application context. Moving from theoretical understanding of cosine similarity and sparse matrix representations to actually implementing these mathematical operations in production code, optimizing them for memory efficiency, and observing their performance characteristics under realistic load conditions provided a depth of understanding that purely academic study cannot replicate. The Docker containerization experience illuminated the genuine complexity that underlies seemingly simple deployment operations. The GitHub Actions CI/CD pipeline experience demonstrated the transformative impact that deployment automation has on development velocity and operational confidence, validating every architectural decision made throughout the project.

CHAPTER 6 – CONCLUSION AND FUTURE ENHANCEMENTS

6.1 CONCLUSION

The CineMatch project represents a comprehensive and successful application of modern software engineering principles to a problem of genuine practical significance. By engineering a platform that provides mathematically intelligent, security-conscious, and operationally efficient movie recommendations, the project directly addresses the real and growing problem of content discovery fatigue in the digital streaming era. The Hybrid Machine Learning approach proved highly effective at overcoming the individual limitations of its constituent algorithms. The dynamic weight adjustment mechanism successfully bridged the cold-start gap, ensuring that users receive relevant recommendations from their very first interaction. The cloud-native microservice architecture deployed on Microsoft Azure validated the feasibility of operating sophisticated machine learning applications within severe resource and cost constraints, achieving a zero-dollar monthly operational cost through careful service selection, multi-stage Docker optimization, and in-memory caching strategies.

6.2 FUTURE SCOPE

Planned enhancements include:

- Neural Collaborative Filtering: Migration of the core recommendation engine to Neural Collaborative Filtering implemented using TensorFlow or PyTorch, capturing complex non-linear relationships between user preferences and item characteristics.
- Real-Time Recommendation Updates: Integration of WebSocket connections to push updated recommendations to the client automatically as soon as a new rating is processed.
- Kubernetes Scaling: Transition from Azure App Service to Azure Kubernetes Service for fine-grained horizontal scaling, accompanied by migration from SQLite to Azure Database for PostgreSQL.
- Contextual Signals: Incorporation of signals such as time of day, day of week, current season, and device type to enable context-aware recommendations.
- Enhanced Kids Zone: Integration of natural language processing models to analyze movie synopses for more nuanced content classification beyond metadata-based filtering.
- Social Features: Introduction of opt-in social discovery features allowing users to share watchlists and view aggregate recommendations from trusted friends.

REFERENCES

[1] Aggarwal, C. C. (2016). Recommender Systems: The Textbook. Springer International Publishing.

[2] Microsoft Azure Documentation. (2025). Azure App Service Overview. Microsoft Corporation.

[3] Grinberg, M. (2018). Flask Web Development: Developing Web Applications with Python. O'Reilly Media.

[4] Docker Documentation. (2025). Use Multi-Stage Builds. Docker Incorporated.

[5] Scipy Community. (2025). Sparse Matrices Documentation (scipy.sparse). SciPy Documentation Portal.

[6] Burke, R. (2002). Hybrid Recommender Systems: Survey and Experiments. User Modeling and User-Adapted Interaction, 12(4), 331–370.

[7] Koren, Y., Bell, R., and Volinsky, C. (2009). Matrix Factorization Techniques for Recommender Systems. IEEE Computer, 42(8), 30–37.

[8] Lops, P., de Gemmis, M., and Semeraro, G. (2011). Content-Based Recommender Systems: State of the Art and Trends. In Recommender Systems Handbook. Springer.

[9] React Documentation. (2025). Getting Started with React. Meta Platforms Incorporated.

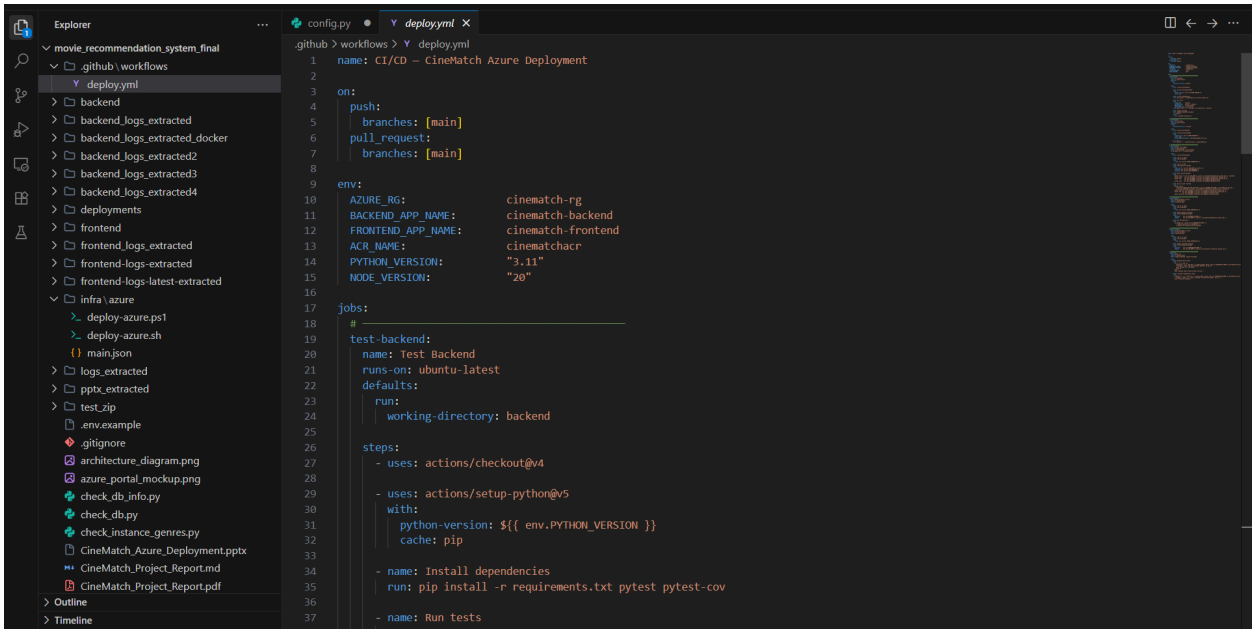
[10] Tailwind CSS Documentation. (2025). Core Concepts: Utility-First Fundamentals. Tailwind Labs.

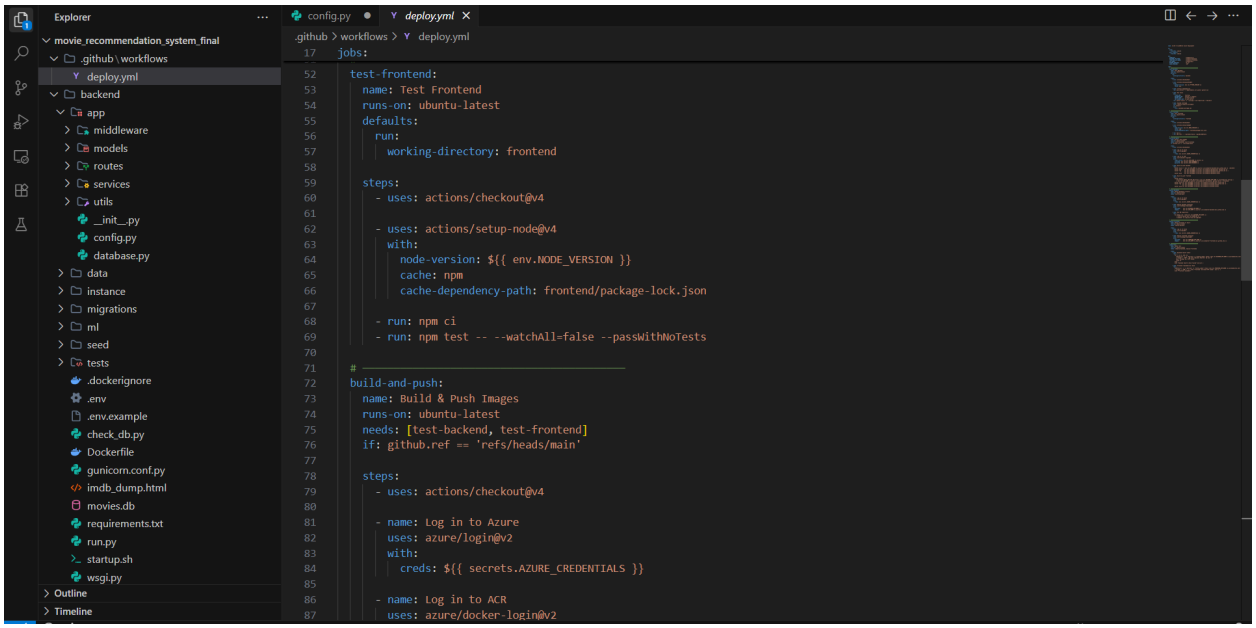
[11] GitHub. (2025). GitHub Actions Documentation. Retrieved from docs.github.com/en/actions.

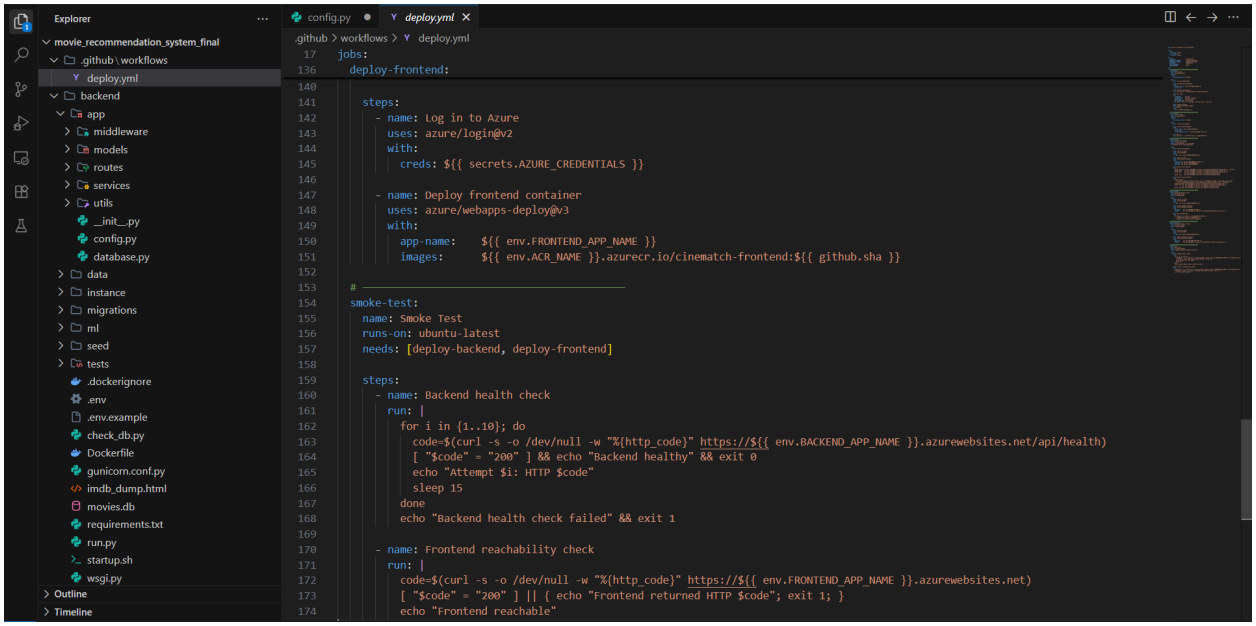
[12] OWASP Foundation. (2025). OWASP Top 10 Web Application Security Risks. Retrieved from owasp.org/www-project-top-ten/.

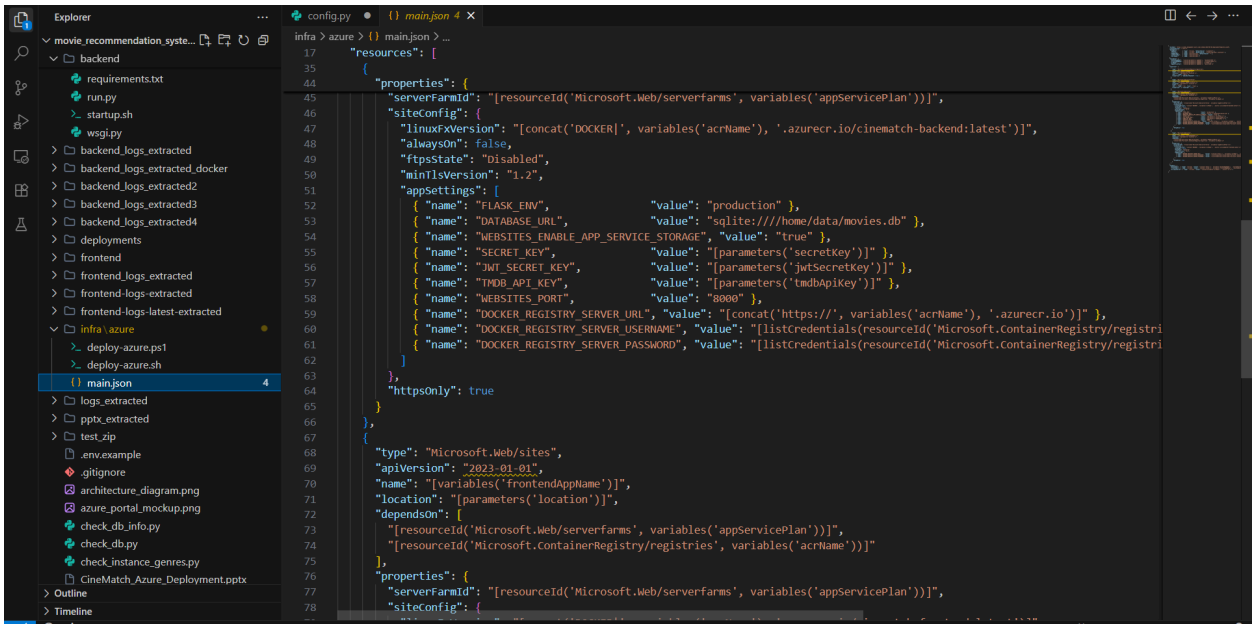
APPENDICES

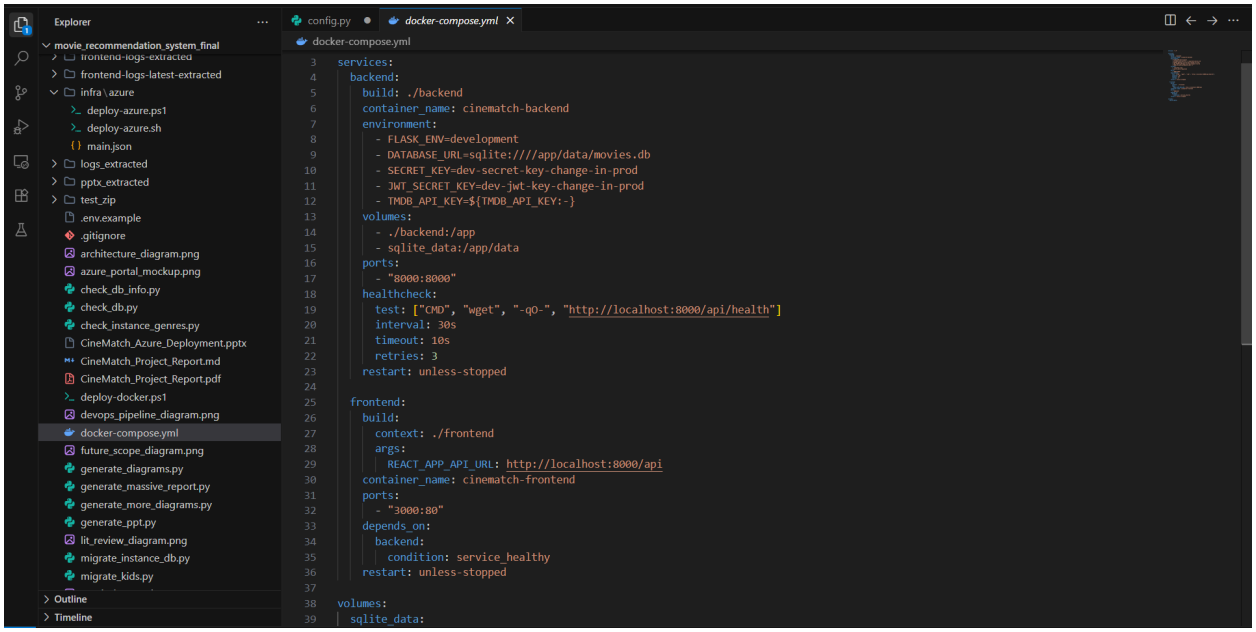
Appendix A – Code Snippets



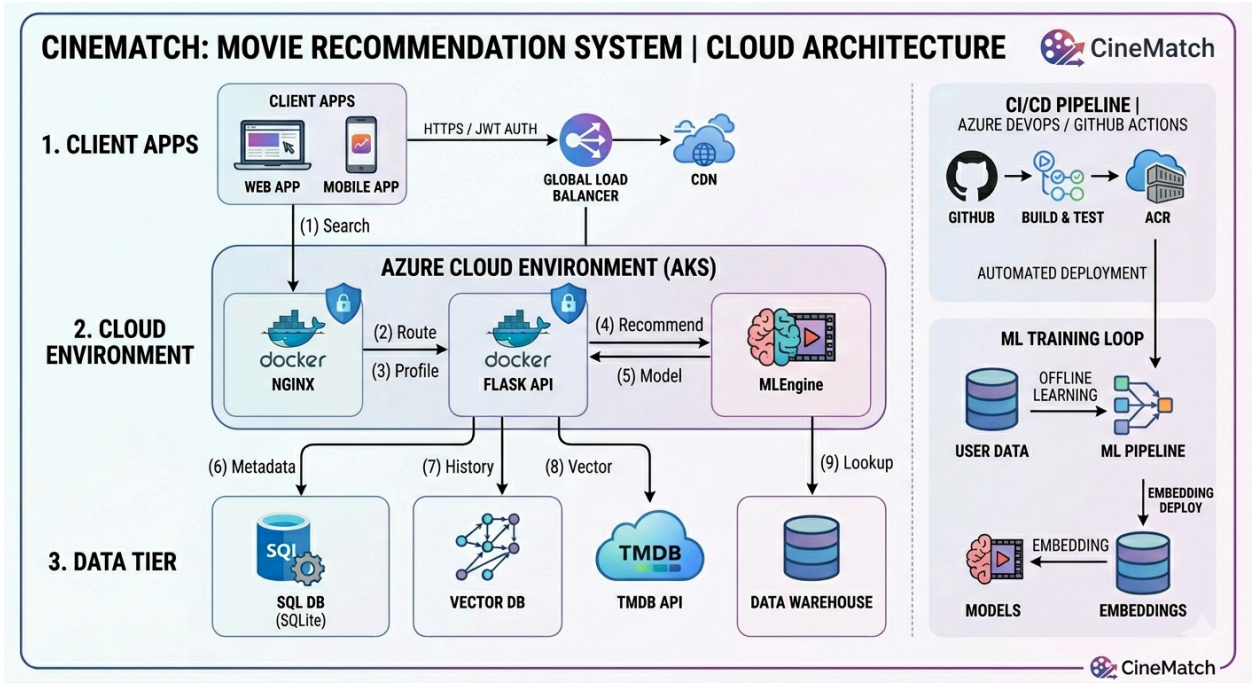



Explorer
movie_recommendation_system_final
.github \ workflows
deploy.yml
backend
app
middleware
models
routes
services
utils
__init__.py
config.py
database.py
data
instance
migrations
ml
seed
tests
.dockerignore
.env
.env.example
check_db.py
Dockerfile
gunicorn.conf.py
imdb_dump.html
movies.db
requirements.txt
run.py
startup.sh
wsgi.py
Outline
Timeline
config.py
deploy.yml
.github > workflows > deploy.yml
jobs:
deploy-frontend:
steps:
- name: Log in to Azure
uses: azure/login@v2
with:
creds: ${{ secrets.AZURE_CREDENTIALS }}
- name: Deploy frontend container
uses: azure/webapps-deploy@v3
with:
app-name: ${{ env.FRONTEND_APP_NAME }}
images: ${{ env.ACR_NAME }}.azurecr.io/cinematch-frontend:${{ github.sha }}
smoke-test:
name: Smoke Test
runs-on: ubuntu-latest
needs: [deploy-backend, deploy-frontend]
steps:
- name: Backend health check
run: |
for i in {1..10}; do
code=$(curl -s -o /dev/null -w "%{http_code}" https://${{ env.BACKEND_APP_NAME }}.azurewebsites.net/api/health)
["$code" = "200"] && echo "Backend healthy" && exit 0
echo "Attempt $i: HTTP $code"
sleep 15
done
echo "Backend health check failed" && exit 1
- name: Frontend reachability check
run: |
code=$(curl -s -o /dev/null -w "%{http_code}" https://${{ env.FRONTEND_APP_NAME }}.azurewebsites.net)
["$code" = "200"] || { echo "Frontend returned HTTP $code"; exit 1; }
echo "Frontend reachable"

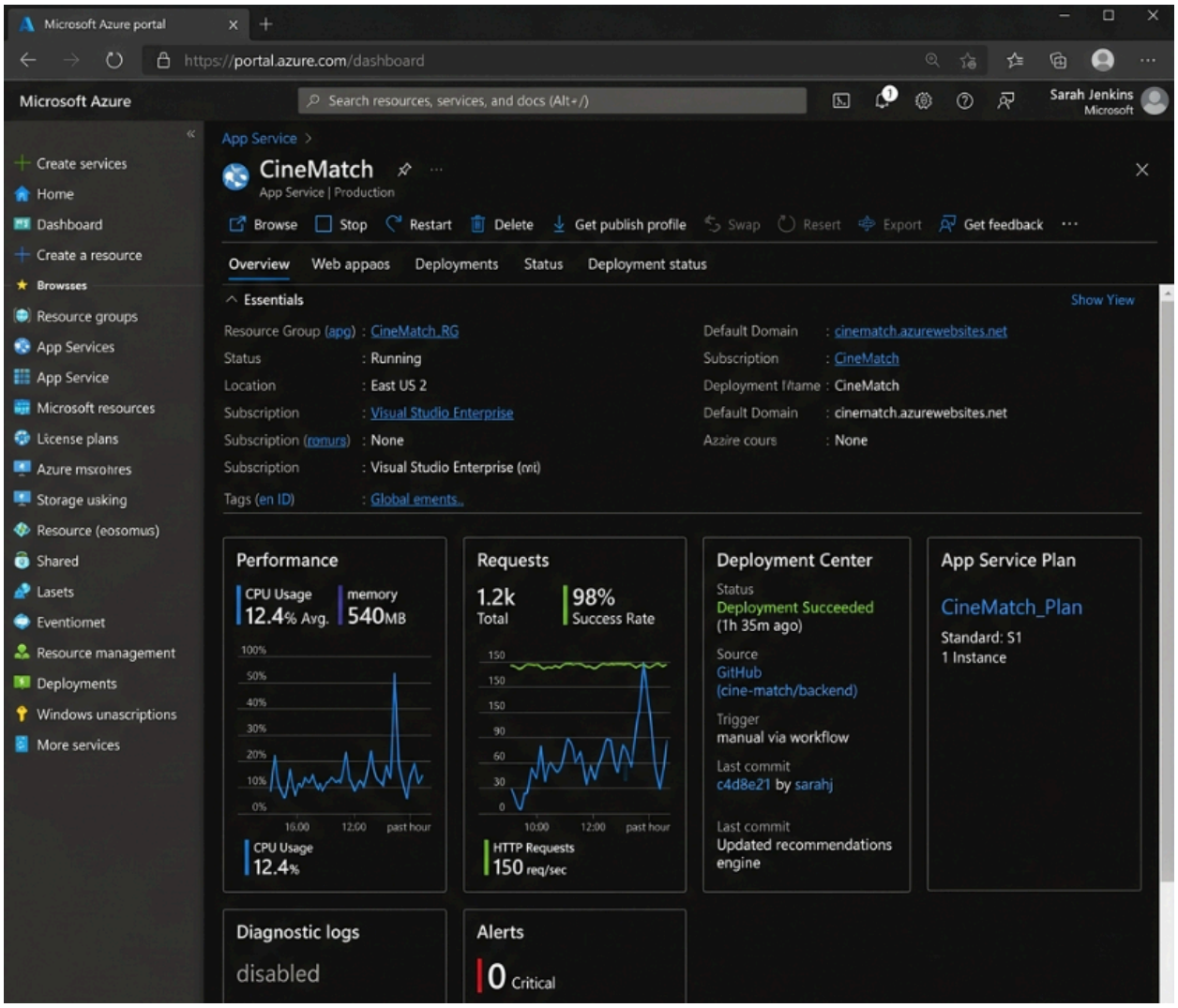

Explorer
movie_recommendation_syste...
backend
requirements.txt
run.py
startup.sh
wsgi.py
backend_logs_extracted
backend_logs_extracted_docker
backend_logs_extracted2
backend_logs_extracted3
backend_logs_extracted4
deployments
frontend
frontend_logs_extracted
frontend-logs-extracted
frontend-logs-latest-extracted
infra \ azure
deploy-azure.ps1
deploy-azure.sh
main.json 4
logs_extracted
pptx_extracted
test_zip
.env.example
.gitignore
architecture_diagram.png
azure_portal_mockup.png
check_db_info.py
check_db.py
check_instance_genres.py
CineMatch_Azure_Deployment.pptx
Outline
Timeline
config.py
main.json 4
infra > azure > main.json > ...
"resources": [
{
"properties": {
"serverFarmId": "[resourceId('Microsoft.Web/serverfarms', variables('appServicePlan'))]",
"siteConfig": {
"linuxFxVersion": "[concat('DOCKER|', variables('acrName'), '.azurecr.io/cinematch-backend:latest')]",
"alwaysOn": false,
"ftpsState": "Disabled",
"minTlsVersion": "1.2",
"appSettings": [
{ "name": "FLASK_ENV", "value": "production" },
{ "name": "DATABASE_URL", "value": "sqlite:////home/data/movies.db" },
{ "name": "WEBSITES_ENABLE_APP_SERVICE_STORAGE", "value": "true" },
{ "name": "SECRET_KEY", "value": "[parameters('secretKey')]" },
{ "name": "JWT_SECRET_KEY", "value": "[parameters('jwtSecretKey')]" },
{ "name": "TMDB_API_KEY", "value": "[parameters('tmdbApiKey')]" },
{ "name": "WEBSITES_PORT", "value": "8000" },
{ "name": "DOCKER_REGISTRY_SERVER_URL", "value": "[concat('https://', variables('acrName'), '.azurecr.io')]" },
{ "name": "DOCKER_REGISTRY_SERVER_USERNAME", "value": "[listCredentials(resourceId('Microsoft.ContainerRegistry/registri
{ "name": "DOCKER_REGISTRY_SERVER_PASSWORD", "value": "[listCredentials(resourceId('Microsoft.ContainerRegistry/registri
]
},
"httpsOnly": true
}
},
{
"type": "Microsoft.Web/sites",
"apiVersion": "2023-01-01",
"name": "[variables('frontendAppName')]",
"location": "[parameters('location')]",
"dependsOn": [
"[resourceId('Microsoft.Web/serverfarms', variables('appServicePlan'))]",
"[resourceId('Microsoft.ContainerRegistry/registries', variables('acrName'))]"
],
"properties": {
"serverFarmId": "[resourceId('Microsoft.Web/serverfarms', variables('appServicePlan'))]",
"siteConfig": {



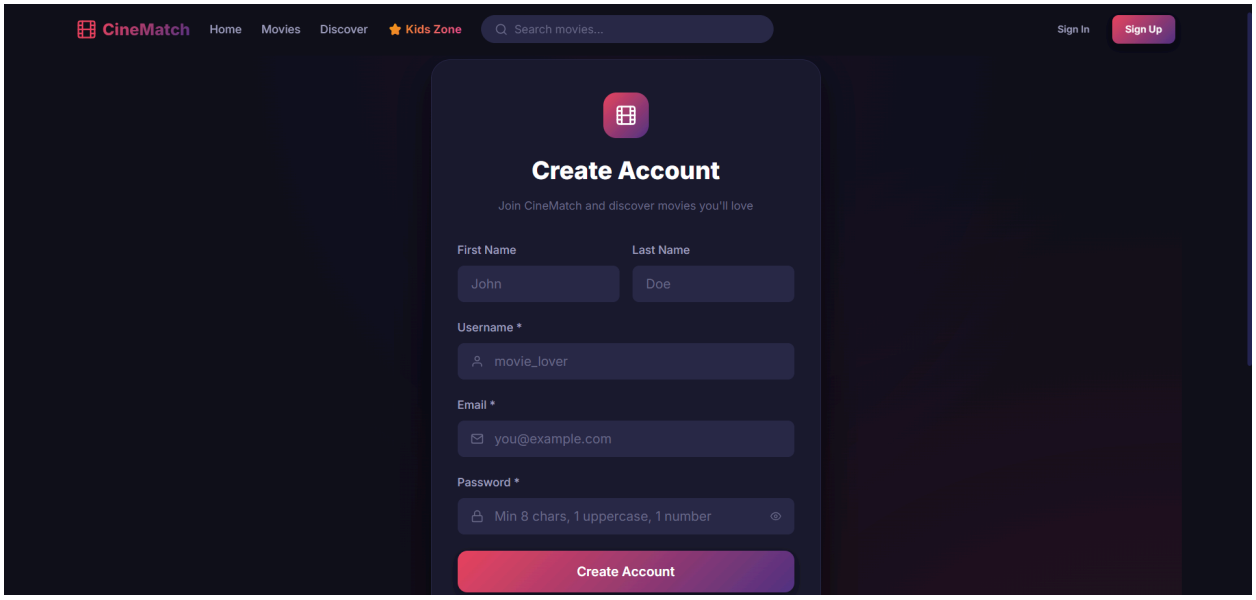
Appendix B – Cloud Services Workflow

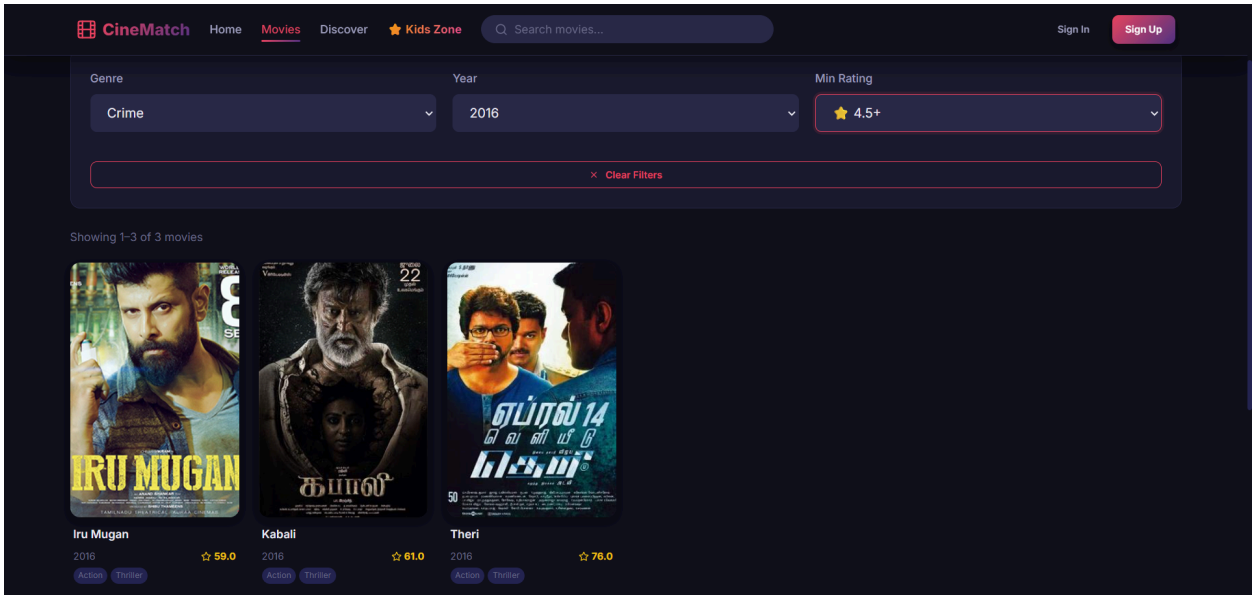


Appendix C – Azure Portal Deployment Evidence



Appendix D – Application Screenshots (Deployment)





CineMatch Home Movies Discover Kids Zone Search movies... Sign In Sign Up

Comedy Drama Romance

Nanban

2012 2h 49m ★ 7.0 (100 ratings)

★★★★★ 7.00 / 5.00

Overview

A remake of 3 Idiots following three engineering students and their professor whose unconventional teaching changes their lives.

Actor	Vijay
Director	Shankar
Runtime	2h 49m
Year	2012
Language	Tamil
Genres	Comedy, Drama, Romance
Status	Released
Budget	₹45 Cr

RATE THIS MOVIE ★★★★★

Add to Watchlist

CineMatch Home Movies Discover Kids Zone Search movies... Watchlist 3 B

My Watchlist

3 movies

Dhruva Natchathiram

2023 ★ 70.0

Action Thriller

Remove

Kanaa

2018 ★ 80.0

Comedy Drama

Remove

Vikram Special Edition

2023 ★ 78.0

Action Thriller

Remove

Appendix E – Environment Configuration Reference

Table 8: Environment Configuration Variables

Variable Name	Purpose
FLASK_SECRET_KEY	Cryptographically random string for Flask session signing (32+ chars)
JWT_REFRESH_SECRET_KEY	Separate key for refresh token signing and rotation
TMDB_API_KEY	API key obtained from the TMDB developer portal
DATABASE_URL	Path to the SQLite database file on the persistent volume
FRONTEND_ORIGIN_URL	Frontend Azure deployment URL for CORS whitelist configuration
CACHE_MAX_SIZE	Maximum number of user recommendation lists held in memory
CACHE_TTL_SECONDS	Time-to-live duration for cached recommendation entries